池州学院·正常模式 · 答案与解析

一、单项选择题（5×2=10分）

1.

A. -43

参数

- 补码位数 n = 7
- 补码机器数 = 1010101（高位 → 低位）
- 符号位 $b_6 = 1$（负补码），数值位 $b_5 \sim b_0 = 010101$

补码真值定义

对 n 位补码 x = b_{n-1} b_{n-2} … b_0，其十进制真值为：

> 真值 = (−b_{n-1}) × 2^(n−1) + Σ(b_i × 2^i, i=0..n−2)

即符号位取负权值 −2^(n−1)，其余位按正权值展开。

逐位展开（对应 gen.py PROCESS 数据流）

位序(bit)	bit_pos	二进制值	权值	贡献
0	6	1	-2^6	−64
1	5	0	$+2^5$	0
2	4	1	$+2^4$	16
3	3	0	$+2^3$	0
4	2	1	$+2^2$	4
5	1	0	$+2^1$	0
6	0	1	$+2^0$	1

求和：−64 + 16 + 4 + 1 = **−43**

> gen.py PROCESS_SUM 验证：sum = −43 == A = −43 √

结论

正确真值 = −43，对应选项 **A**。

选项	数值	错误类型	说明
A	−43	正确	按补码正确定义展开：$-2^6+2^4+2^2+2^0 = -43$
B	21	符号位判断错误	忽略符号位负权值，仅低 6 位 b_5~b_0 按正权展开：$2^4+2^2+2^0 = 16+4+1 = 21$
C	85	全部按无符号正权展开	将 7 位全部视为正权值：$2^6+2^4+2^2+2^0 = 64+16+4+1 = 85$
D	−42	某一位漏算	按补码展开但漏算 2^0 位（A + 2^0 = −43 + 1 = −42），数值近邻

2.

\boxed{A. 0xC1BD8000}

> 交叉验证：使用 Python `struct.pack('>f', -23.6875)` 得到 0xC1BD8000，与手算结果一致。

3.

答案：A

解析：直接映射方式下，主存中的每一块只能映射到 Cache 中唯一指定的一行，映射关系由"主存块号 mod Cache 行数"确定，即取模运算。A 正确。

- B 描述的是全相联映射的特征（任意块可映射到任意行），与直接映射矛盾。
- C 描述的是组相联映射的特征（组间直接映射、组内全相联），张冠李戴。
- D 说用"除法"确定映射关系，应为取模（mod）运算，概念错误。

4.

答案：A

解析：基址寻址的有效地址 = 基址寄存器内容 + 形式地址（偏移量）。基址寄存器的内容由操作系统管理（面向多道程序逻辑地址到物理地址的重定位），形式地址作为偏移量在程序运行中不变。A 正确。

- B 描述的是相对寻址（PC + 形式地址），混淆了基址寄存器与 PC。
- C 描述的是变址寻址（变址寄存器 + 形式地址），混淆了基址与变址：基址寄存器由 OS 管理面向重定位，变址寄存器由用户管理面向数组循环。
- D 描述的是立即寻址，形式地址即操作数本身，完全不同。

5.

答案：A

解析：中断隐指令（硬件自动完成的操作）包含三步：①关中断（防止在保存断点时被新中断打断）；②保存断点（将 PC 和 PSW 压入堆栈）；③引出中断服务程序入口地址（将中断向量地址装入 PC）。A 正确。

- B 错误：中断隐指令是关中断而非开中断；保存所有通用寄存器是中断服务程序（软件）的工作，不是隐指令完成的。
- C 错误：保存所有通用寄存器不是隐指令的工作，属于中断服务程序的任务；且隐指令完成后 PC 已指向中断服务程序入口，而非"直接执行第一条指令"这一步。
- D 错误：隐指令第一步是关中断而非开中断；中断服务程序入口地址装入 PC 而非通用寄存器。

二、判断题（5×2=10分）

1.

答案：×（错）

解析：错误。SRAM 存取确实快于 DRAM，但 SRAM 每位用双稳态触发器（元件多）、集成度**低于** DRAM；DRAM 每位仅 1 个 MOS 管+电容，集成度更高。

2.

答案：√（对）

解析：正确。虚存容量取决于虚拟地址位数（程序员可见地址空间），实际运行由辅存支撑，故逻辑地址空间可大于物理内存。

3.

答案：√（对）

解析：正确。链式查询优先级由物理位置决定（近高远低）；链路上某设备故障会使其后所有设备都无法获总线授权，故对故障敏感。

4.

答案：×（错）

解析：错误。DMA 仅在**数据传送阶段**由 DMA 控制器接管总线、CPU 不介入；但**预处理**（设置传送参数）与**后处理**（计数/校验/中断通知）仍由 CPU 完成。

5.

答案：√（对）

解析：正确。流水线分段重叠执行，稳态下每周期流出一条指令；实际因结构/数据/控制冲突停顿而低于该理想值。

三、计算题（40分）

1.

变形补码加减运算与溢出判断 —— 参考解答

基本信息

- 编码：9 位变形补码（2 位符号位 + 7 位数值位），表示范围 [−128, +127]
- x = +95，y = +73
- 溢出判断规则：双符号位相同（00/11）不溢出；不同（01/10）溢出。符号位参与运算。

第（1）问：x 和 y 的变形补码表示

x = +95

- 绝对值 7 位二进制：|95| = 95 = `101 1111`，即 7 位为 **1011111**
- 正数双符号位为 **00**
- **[x]变补 = 00 1011111 = 001011111**（9 位）

y = +73

- 绝对值 7 位二进制：|73| = 73 = `100 1001`，即 7 位为 **1001001**
- 正数双符号位为 **00**
- **[y]变补 = 00 1001001 = 001001001**（9 位）

第（2）问：变形补码计算 x + y

变形补码加法竖式

```
  [x]变补 = 001011111
+ [y]变补 = 001001001
──────────────────
  结果    = 010101000   (整数值 = 168)
```

双符号位判断

- 运算结果 = `010101000`
- 双符号位 = **01**（两符号位不同）
- **双符号位判断：溢出（正溢出）**

真值范围验证

- 真值 x + y = 95 + 73 = **+168**
- +168 > +127，超出 9 位变形补码表示范围 [−128, +127]
- 真值范围判断：溢出

结论

x + y 溢出。 原因：x 和 y 同为正数，两数之和真值为 +168，超出 7 位数值位变形补码的表示范围 [−128，+127]。变形补码加法结果的双符号位为 01（正溢出标志），双符号位判断与真值范围判断一致——正溢出。

第（3）问：变形补码计算 x − y

第一步：求 [−y] 的变形补码

对 [y]变补 连同符号位一起取反加一：

```
[y]变补      = 001001001
取反         = 110110110
取反加一     = 110110111
```

- **[−y]变补 = 110110111**（整数值 = 439）
- 验证：[−y]变补 对应真值 = 439 − 512 = **−73** √

第二步：变形补码加法竖式 [x]变补 + [−y]变补

```
  [x]变补   = 001011111
+ [-y]变补  = 110110111
──────────────────
  结果      = 000010110   (整数值 = 22)
```

双符号位判断

- 运算结果 = `000010110`
- 双符号位 = **00**（两符号位相同）
- **双符号位判断：不溢出**

真值范围验证

- 真值 x − y = 95 − 73 = **+22**
- +22 ∈ [−128, +127]，在表示范围内
- 真值范围判断：不溢出
- 结果变形补码 `000010110` 对应真值 = **+22** √

结论

x − y 不溢出。双符号位为 00（相同），运算结果真值为 **+22**，在 [−128, +127] 范围内。双符号位判断与真值范围判断一致——不溢出。

汇总

运算	变形补码结果	双符号位	双符号位判断	真值	真值范围判断	结论
x + y	010101000	01	溢出	+168	超出[−128,+127]	**溢出（正溢出）**
x − y	000010110	00	不溢出	+22	在[−128,+127]内	**不溢出，真值 +22**

- x + y 与 x − y 中恰好一组溢出、一组不溢出，符合题目设计。
- 两组运算的双符号位判断法与真值范围判断法完全一致。

2.

$\boxed{Z = 0x408B8000}$

- 最终阶码 = **2**（十进制），阶码字段 = **10000001**（二进制，129 十进制）
- 规格化尾数（1.M）= **1.00010111000000000000000**，23 位尾数字段 = **00010111000000000000000**

3.

> 本解答所有数值由 gen.py exec 产出，逐字对照数据流输出转写。

第 1 问（3 分）

(1) 主存地址划分

地址总位数 = 12 位，按字节编址。

字段	位数	推导
块内偏移（Offset）	**5 位**	块大小 32 B = 2^5 B → 偏移 5 位
Cache 行号（Index）	**3 位**	Cache 8 块 = 2^3 → 行号 3 位（直接映射）
标记（Tag）	**4 位**	12 − 5 − 3 = 4 位

地址格式：**Tag(4) | Index(3) | Offset(5)**

地址守恒验证：4 + 3 + 5 = 12 ✓

(2) 逐次访存分析

地址解析方法：Index = (addr >> 5) & 0b111，Tag = addr >> 8。

序号	地址	12 位二进制	Tag	Index	命中/缺失	说明
1	0x040	000001000000	0	2	**M**	Cache 空，缺失，装入行 2（Tag=0）
2	0x080	000010000000	0	4	**M**	行 4 空，缺失，装入行 4（Tag=0）
3	0x040	000001000000	0	2	**H**	行 2 有效且 Tag=0 匹配，命中
4	0x0C0	000011000000	0	6	**M**	行 6 空，缺失，装入行 6（Tag=0）
5	0x080	000010000000	0	4	**H**	行 4 有效且 Tag=0 匹配，命中
6	0x140	000101000000	1	2	**M**	**冲突缺失**：行 2 有效但 Tag=0 ≠ 1，替换装入 Tag=1
7	0x040	000001000000	0	2	**M**	**冲突缺失**：行 2 现为 Tag=1（被第 6 次替换），0x040 被替换掉 → 缺失
8	0x0C0	000011000000	0	6	**H**	行 6 有效且 Tag=0 匹配，命中

命中次数 = 3，缺失次数 = 5，总访问 = 8。

冲突缺失 = 2 次（第 6、7 次）：第 6 次访问 0x140（Tag=1，Index=2）与第 1/3 次访问 0x040（Tag=0, Index=2）映射到同一 Cache 行，0x040 被替换，导致第 7 次访问 0x040 变为缺失。

> **陷阱提示**：第 7 次 0x040 看似与第 1/3 次相同，易误判为命中；但第 6 次 0x140 的冲突已将 0x040 从 Cache 中替换出去，故第 7 次为缺失。

(3) Cache 命中率

h = \frac{命中次数}{总访问次数} = 3/8

第 2 问（2 分）：平均访问时间

缺失代价定义：缺失时先访问 Cache 确认未命中（t_c），再访问主存取块（t_m'），故：

缺失代价 = $t_c + t_m' = 2 + 18 = 20$ ns

$t_a = h \cdot t_c + (1 - h) \cdot (t_c + t_m')$

$t_a = 3/8 \times 2 + 5/8 \times 20 = 6/8 + 100/8 = 106/8 = 53/4 = 13.25$ ns

\boxed{$t_a = 13.25$ ns}

> **陷阱提示**：缺失代价须包含 Cache 访问时间 t_c（题干明确"先访问 Cache 确认未命中"），不能仅取 t_m'。

第 3 问（2 分）：加速比

无 Cache 时每次访存均直接访问主存，单次访存时间 = $t_c + t_m'$。

无 Cache 单次访存时间 = $t_c + t_m' = 2 + 18 = 20$ ns

$S = t_c + t_m'/t_a = \frac{20}{53/4} = 80/53 \approx 1.51$

$\boxed{S \approx 1.51}$

场景规则验证汇总

规则	验证结果
Cache 容量一致性	$32 \times 8 = 256$ B √
地址位数守恒	$4 + 3 + 5 = 12$ √
命中率非平凡	$h = 3/8$，$0 < h < 1$ √
平均时间公式	$t_a = h \cdot t_c + (1-h) \cdot (t_c + t_m')$ √
加速比 > 1	$S \approx 1.51 > 1$ √
冲突缺失存在	冲突缺失 2 次 $\geqslant 1$ √
时间参数约束	$t_c + t_m' = 20 \geqslant 5 \times t_c = 10$ √

4.

扩展操作码各级指令最大条数 — 求解

已知参数

参数	值	含义
L	16 位	指令字长
W	4 位	地址码字段宽度
N3	14 条	三地址指令已定义条数

第 1 问（2 分）：三地址操作码位宽与预留编码数

操作码位宽

三地址指令有 3 个地址码字段，每个 W = 4 位，地址码共占 3 × 4 = 12 位。操作码位于高位：

操作码位宽 = L - 3 × W = 16 - 3 × 4 = 16 - 12 = \mathbf{4 位}

预留编码数

4 位操作码共有 2^4 = 16 个编码模式。已知三地址指令定义了 14 条，因此可用于扩展到二地址的预留编码数为：

预留编码数 = 2^4 - 14 = 16 - 14 = \mathbf{2 个}

> 答：三地址操作码字段宽度为 **4 位**；预留 **2** 个编码模式用于向二地址扩展。

第 2 问（2 分）：二地址、一地址最大条数

二地址指令

- **操作码位宽** = L - 2 × W = 16 - 2 × 4 = 8 位
- 三地址预留了 2 个前缀编码，二地址在操作码高位复用这 2 个前缀，再加上自己的 W = 4 位地址码腾出的扩展位，总编码数为：

二地址总编码数 = 三地址预留数 × 2^W = 2 × 2^4 = 2 × 16 = 32

- 为扩展到一地址，需保留 1 个前缀，故最大条数：

二地址最多 = 32 - 1 = \mathbf{31 条}

一地址指令

- **操作码位宽** = L - 1 × W = 16 - 1 × 4 = 12 位
- 二地址保留了 1 个前缀编码，一地址总编码数为：

一地址总编码数 = 二地址预留数 × 2^W = 1 × 2^4 = 1 × 16 = 16

- 为扩展到零地址，需保留 1 个前缀，故最大条数：

一地址最多 = 16 - 1 = \mathbf{15 条}

> 答：二地址指令最多 **31** 条；一地址指令最多 **15** 条。

第 3 问（3 分）：零地址操作码位宽与最大条数

操作码位宽

零地址指令无地址码字段：

操作码位宽 = L - 0 × W = 16 - 0 = \mathbf{16 位}

零地址最大条数 — 完整逐级推导链

从三地址的预留编码出发，逐级推导：

级别	前缀来源	总编码数计算	保留前缀	最大条数
三地址	4 位操作码	$2^4 = 16$	16 - 14 = 2	14（已知）
二地址	三地址预留 2 个	$2 \times 2^4 = 32$	1	32 - 1 = 31
一地址	二地址预留 1 个	$1 \times 2^4 = 16$	1	16 - 1 = 15
零地址	**一地址预留 1 个**	$1 \times 2^4 = 16$	**0（最后一级）**	**16**

零地址是最后一级，无需保留前缀编码，因此最大条数等于总编码数：

零地址最多 = 一地址预留数 × 2^W = 1 × 2^4 = 1 × 16 = \mathbf{16 条}

> **答**：零地址操作码字段宽度为 **16 位**；零地址指令最多 **16** 条。

计算依据小结

1. **操作码位宽公式**：k 地址指令的操作码位宽 = L - k × W。
2. **预留前缀机制**：每级操作码总编码数 $2^{\text{位宽}}$ 减去已定义指令数，剩余即为向下扩展的预留编码数。
3. **下一级总编码数** = 上一级预留数 × 2^W（上级腾出的扩展位 × 本级地址码腾出的 4 位）。
4. **非最后级**最大条数 = 总编码数 - 1（留 1 个前缀给更下一级）；**零地址**（最后一级）最大条数 = 总编码数（无需保留）。

级别	操作码位宽	总编码数	预留前缀	最大条数
三地址	4 位	16	2	14
二地址	8 位	32	1	31
一地址	12 位	16	1	15
零地址	16 位	16	—	16

5.

子题	答案
第（1）题	T_pipeline = **320 ns**，TP = **0.1875 条/ns**

子题	答案
第（2）题	$S = 3.5625$
第（3）题（a）	$TP' \approx 0.2308$ 条/ns
第（3）题（b）	吞吐率提升约 **23.08%**，原因是瓶颈段拆分使时钟周期 5→4 ns

6.

总线带宽计算 —— 求解

> 本文档逐字对照 gen.py exec 数据流转写，数值均来自 exec 输出。

已知参数

参数	值
数据线宽度 W	4 B（32 位）
总线时钟频率 f	150 MHz = 1.5×10^8 Hz
事务周期数 N	3
待传输数据量 D	4097 B

（1）理论带宽（2 分）

理论带宽 = 数据线宽度 × 时钟频率（每个时钟周期传输 W 字节）

理论带宽 = W × f = 4 B × 1.5×10^8 Hz = 6.0×10^8 B/s

换算为 MB/s（1 MB = 10^6 B）：

理论带宽 = $6.0 \times 10^8/10^6 = \boxed{600 \text{ MB/s}}$

（2）有效带宽（2 分）

每个总线事务需要 N = 3 个时钟周期才能完成一次数据传输，因此有效带宽降为理论的 1/N：

有效带宽 = $\frac{\text{理论带宽}}{N} = 6.0 \times 10^8/3 = 2.0 \times 10^8$ B/s

换算为 MB/s：

有效带宽 = $2.0 \times 10^8/10^6 = \boxed{200 \text{ MB/s}}$

（3）传输 4097 B 数据所需时间（3 分）

第一步：计算传输次数（向上取整）

每次事务传输 W = 4 B。4097 B 不能被 4 整除（4097 \div 4 = 1024.25，4097 \bmod 4 = 1 ≠ 0），最后不足 4 B 的部分仍需一次事务传完，因此必须向上取整：

传输次数 = \lceil D / W \rceil = \lceil 4097 / 4 \rceil = \lceil 1024.25 \rceil = 1025 次

第二步：计算总周期数

每次事务占 N = 3 个时钟周期：

总周期数 = 1025 × 3 = 3075 个周期

第三步：计算总时间

时间 = \frac{总周期数}{f} = 3075/1.5 × 10^8 = 2.05 × 10^{-5} s

换算为 μs（1 s = 10^6 μs）：

时间 = 2.05 × 10^{-5} × 10^6 = \boxed{20.5 μs}

结果汇总

子问	结果
（1）理论带宽	**600 MB/s**
（2）有效带宽	**200 MB/s**
（3）传输时间	**20.5 μs**

四、综合题（4×10=40分）

1.

位扩展（扩展数据位宽）：

CPU 数据线 D = 8 位，芯片字宽 W = 4 位，需要：

位扩展数 = D/W = 8/4 = 2 片/组

（D % W = 8 % 4 = 0，整除，位扩展为整数）

字扩展（扩展存储字数）：

CPU 寻址空间 = 2^{16}，芯片地址空间 = 2^{14}，需要：

字扩展组数 = 2^{A-K} = 2^{16-14} = 2^2 = 4 组

（A − K = 16 − 14 = 2 ⩾ 1，字扩展非平凡）

芯片总数：

总片数 = 位扩展数 × 字扩展组数 = 2 × 4 = \boxed{8 片}

问题	答案
Q1	位扩展 2 片 × 字扩展 4 组 = **8 片**
Q2	片内寻址 14 根（A13~A0），片选 2 根（A15~A14）
Q3	A15A14 经 2-4 译码器产生 4 个片选信号；组内两片并联片选+地址，数据分接高低 4 位
Q4	128 行；刷新间隔 16 μs；刷新次数 128 次

2.

指令执行微操作序列与控制信号——参考解答

基本参数

参数	值
编址方式	按字节编址
地址线位数	16 位
总线宽度	16 位
存储字长	16 位
指令字长	16 位 = 2 字节
PC 增量（取指后）	**PC + 2**

> 控制信号清单：PCin、PCout、MARin、MDRin、MDRout、IRin、R0in–R7in、R0out–R7out、Yin、Zin、Zout、ALU_ADD、ALU_SUB、ALU_AND、ALU_OR、Read、Write。

第 1 问（3 分）：取指周期微操作序列

节拍	微操作	控制信号
T1	PC → MAR	PCout, MARin
T2	PC + 2 → PC	PCout, Yin, ALU_ADD, Zin, Zout, PCin
T3	Mem[MAR] → MDR → IR	Read, MDRout, IRin

说明： - T1：将 PC 内容送 MAR，选中指令所在的主存单元。 - T2：PC 通过 ALU 完成 +2 运算（PC→Y 锁存器→ALU A 端，常数 2 经总线→ALU B 端，ALU_ADD，结果存 Z 锁存器，Z→总线→PC 回写）。按字节编址、指令字长 2 字节，故 PC 增量为 2。 - T3：发出 Read 信号，主存将指令读入 MDR，再由 MDR 经总线送入 IR。

第 2 问（4 分）：LDR R1, @40 间址周期 + 执行周期

LDR R1, @40 采用**间接寻址**：形式地址 40 是主存单元地址，该单元的内容才是操作数的有效地址 EA。因此取指后需要**间址周期**取出有效地址，再进入**执行周期**取操作数。

间址周期（取有效地址 EA）

节拍	微操作	控制信号
T1	IR 地址字段（形式地址 40）→ MAR	MARin
T2	Mem[MAR] → MDR	Read
T3	MDR（= 有效地址 EA）→ MAR	MDRout, MARin

说明： - T1：IR 中形式地址 40（经控制器内部通路）送入 MAR，指向存放有效地址的主存单元。 - T2：发出 Read 信号，读出主存[40] 的内容（即有效地址 EA）到 MDR。 - T3：将 MDR 中的有效地址 EA 送回 MAR，为执行周期访存做准备。

执行周期（按 EA 取操作数 → R1）

节拍	微操作	控制信号
T1	Mem[MAR]（EA 处）→ MDR	Read
T2	MDR → R1	MDRout, R1in

说明： - T1：此时 MAR 中已是有效地址 EA，发出 Read 信号，读出操作数到 MDR。 - T2：MDR 中的操作数经总线送入目的寄存器 R1，完成装入。

第 3 问（3 分）：ADD R2, R3 执行周期 + 对比分析

ADD R2, R3 采用**寄存器直接寻址**，两操作数均在寄存器中，不需要访存。

执行周期微操作序列

节拍	微操作	控制信号
T1	R2 → Y 锁存器	R2out, Yin

节拍	微操作	控制信号
T2	R3 → 总线 → ALU B 端；Y → ALU A 端；ADD → Z	R3out, ALU_ADD, Zin
T3	Z → 总线 → R2	Zout, R2in

说明： - T1：第一操作数 R2 的内容经总线送入 Y 锁存器（ALU 的 A 端输入）。 - T2：第二操作数 R3 的内容经总线直接送入 ALU 的 B 端，与 Y 锁存器中的 A 端数据执行加法（ALU_ADD），结果存入 Z 锁存器（Zin）。 - T3：Z 锁存器中的运算结果经总线写回目的寄存器 R2。

> 该序列是单总线数据通路描述下的**唯一合理微操作安排**：单总线同一时刻只能有一个部件输出，因此两个源操作数必须分两步——先送一个到 Y 锁存器，再送另一个到总线——这不存在二义性。

ADD 不经过访存控制信号（Read/Write）的原因

ADD 指令的执行周期控制信号集合为：**{R2out, Yin, R3out, ALU_ADD, Zin, Zout, R2in}，不含 Read 或 Write**。

原因：ADD R2, R3 的两个操作数分别在 R2 和 R3 中，运算结果也写回 R2，整个过程的数据通路路径为 **寄存器 → 总线 → ALU → 总线 → 寄存器**，全部在 CPU 内部完成，不需要访问主存。而 LDR R1, @40 的操作数存放在主存中，必须通过 MAR/MDR 并发出 Read 信号才能取得操作数，因此 LDR 的执行过程涉及访存控制信号。

验证结论

验证项	结果
PC 增量 = 指令字节数（2）	✅ PASS
数据通路完整性（所有微操作信号均有支撑）	✅ PASS（无支撑信号数 = 0）
ADD 执行序列唯一确定	✅ PASS
控制信号覆盖性（所有信号 ⊆ 清单）	✅ PASS
ADD 不含 Read/Write	✅ True
LDR 含 Read	✅ True

3.

> 本解基于 gen.py 的 exec 输出逐字转写，所有数值来自程序计算结果。

已知参数

指令类别	频度	M1 CPI	M2 CPI
算术逻辑（A）	40%	2	1
访存（B）	25%	4	5
分支跳转（C）	15%	3	2
浮点运算（D）	20%	5	3

· $f_1 = 3.0$ GHz，$f_2 = 2.0$ GHz

频度归一验证：$0.40 + 0.25 + 0.15 + 0.20 = 1.00$ ✓

第 1 问：加权平均 CPI

$CPI_{Mx} = \sum_i p_i \times CPI_{x,i}$

M1：

类别	p_i	$CPI_{1,i}$	贡献
A	0.40	2	0.80
B	0.25	4	1.00
C	0.15	3	0.45
D	0.20	5	1.00

$CPI_{M1} = 0.80 + 1.00 + 0.45 + 1.00 = 3.25$

M2：

类别	p_i	$CPI_{2,i}$	贡献
A	0.40	1	0.40
B	0.25	5	1.25
C	0.15	2	0.30
D	0.20	3	0.60

$CPI_{M2} = 0.40 + 1.25 + 0.30 + 0.60 = 2.55$

> **答：$CPI_{M1} = 3.25$，$CPI_{M2} = 2.55$。**

第 2 问：MIPS 速率

$MIPS_{Mx} = \frac{f_x(Hz)}{CPI_{Mx} \times 10^6} = \frac{f_x(GHz) \times 10^3}{CPI_{Mx}}$

M1：

$MIPS_{M1} = 3.0 \times 10^3 / 3.25 = 923.0769\cdots \approx 923.08$

（Hz 法验证：$3.0 \times 10^9 / 3.25 \times 10^6 = 923.0769$，两种写法一致 √）

M2：

$MIPS_{M2} = 2.0 \times 10^3 / 2.55 = 784.3137\cdots \approx 784.31$

（Hz 法验证：$2.0 \times 10^9 / 2.55 \times 10^6 = 784.3137$，两种写法一致 √）

> **答：$MIPS_{M1} = 923.08$，$MIPS_{M2} = 784.31$。**

第 3 问：哪台机器更快

执行同一程序（IC 相同）时，执行时间为：

$T_{Mx} = \frac{IC \times CPI_{Mx}}{f_x}$

比较执行时间比例因子 CPI / f（IC 为公共常数，略去不比）：

- M1：$CPI_{M1} / f_1 = 3.25 / (3.0 \times 10^9) = 1.0833 \times 10^{-9}$
- M2：$CPI_{M2} / f_2 = 2.55 / (2.0 \times 10^9) = 1.2750 \times 10^{-9}$

M1 的执行时间比例因子更小，故 **M1 执行更快**。

从 MIPS 角度验证（$MIPS \propto 1/T$）：

- $MIPS_{M1} = 923.08$，$MIPS_{M2} = 784.31$

M1 的 MIPS 更高，与执行时间判断一致。

关键说明（暗坑分析）：

本题是经典的"频率不同不能直接比 CPI"陷阱：

- M2 的 CPI（2.55）**低于** M1（3.25），如果**单比 CPI**，会得出"M2 更快"的**错误**结论。
- 但 M2 的主频（2.0 GHz）也**低于** M1（3.0 GHz）——形成 CPI 与频率的**反向权衡**。
- 综合两者后，M1 的 MIPS（923.08）反而高于 M2（784.31），即 M1 执行更快。
- 这说明当两机频率不同时，必须用 MIPS 或执行时间（将 CPI 与主频一起考虑）来判断，不能仅比较 CPI。

> **答：M1 执行更快。虽然 M2 的 CPI 更低（2.55 < 3.25），但 M1 的主频更高（3.0 GHz > 2.0 GHz），综合后 M1 的 MIPS = 923.08 > M2 的 784.31，执行时间更短。单比 CPI 会得出错误结论，必须综合 CPI 与主频。**

第 4 问：改进 M2 后的 MIPS 与加速比

改进方案：B 类指令 CPI 由 5 降至 2.5（原来的 1/2），其余不变。

改进后 M2 的 CPI 表：

类别	p_i	改进后 CPI	贡献
A	0.40	1	0.40
B	0.25	**2.5**	**0.625**
C	0.15	2	0.30
D	0.20	3	0.60

$CPI'_{M2} = 0.40 + 0.625 + 0.30 + 0.60 = 1.925 \approx 1.93$

改进后 MIPS：

$MIPS'_{M2} = 2.0 \times 10^3 / 1.925 = 1038.9610\cdots \approx 1038.96$

加速比：

加速比 = \frac{$MIPS'_{M2}$}{$MIPS_{M2}$} = 1038.96/784.31 = 1.3247⋯ ≈ 1.325

变化幅度验证：(1038.96 - 784.31) / 784.31 = 32.47\% ⩾ 10\% √

> **答：改进后 $MIPS'_{M2} = 1038.96$，加速比 = 1.325。**

4.

问	答案
第 1 问	6144 μs
第 2 问	20.24 μs
第 3 问	99.67%
第 4 问	0.50%